

EvoCrypt: An Upgradable Cryptographic Coprocessor for the Ethereum Virtual Machine

Qi Liu¹, Qianhong Wu¹, and Bo Qin²

¹ School of Cyber Science and Technology, Beihang University, Beijing 100191, China

² School of Information, Renmin University of China, Beijing 100872, China

Abstract. Smart contracts frequently rely on cryptographic algorithms, yet implementing complex cryptographic computation in Solidity expands it into large sequences of EVM instructions, resulting in substantial execution overhead and gas consumption. Precompiled contracts reduce these costs by exposing client-native routines at fixed addresses, but they provide no runtime mechanism for evolving an algorithm independently of client releases and coordinated network upgrades. We present EvoCrypt, an upgradable cryptographic coprocessor that exposes a stable contract-facing interface while executing replaceable WebAssembly modules in a controlled client-side runtime. EvoCrypt distributes version metadata on-chain and deterministically activates an installed module at a designated block height, allowing participating nodes to switch algorithm versions consistently. Each invocation is charged according to a deterministic algorithm-level gas schedule associated with the activated version, thereby avoiding the cost of expanding the algorithm into EVM instructions while retaining explicit resource accounting. We implement EvoCrypt in Geth and evaluate it on a 20 node private Clique network configured with zero-period block production to isolate system-level upgrade overhead from the block interval. Measured *network-wide upgrade duration* is the Geth-side time to finish upgrade handling on all evaluated nodes—from RPC submission through receipt observation, event handling, and node-local WASM preparation—and excludes network propagation and consensus delay; under this definition the prototype completes upgrades within 0.54–2.34 ms across the eight evaluated algorithms. EvoCrypt achieves execution latency comparable to Solidity implementations on lightweight algorithms, with up to 48% lower latency on computation-heavy workloads.

Keywords: blockchain · smart contracts · upgrade · WebAssembly

1 Introduction

As blockchain systems become programmable platforms, smart contracts increasingly depend on cryptographic algorithms for authentication, integrity, signatures, and privacy [12, 24, 20, 6]. The long-term security of these applications also requires the ability to adopt new algorithms, including post-quantum alternatives, without disrupting contract-facing functionality [18, 8, 13]. Blockchain

clients therefore need an extensible way to provide evolving cryptographic capabilities to smart contracts.

Ethereum provides two main mechanisms for executing cryptographic operations: Solidity contract implementations and Precompiled Contracts. Solidity implementations are flexible and deployable at the contract layer, but complex cryptographic computation expands into costly EVM execution; Precompiled Contracts execute selected routines efficiently in client-native code, but their supported algorithms are fixed to the client and normally require coordinated client updates to change [20, 6, 7].

These mechanisms expose a fundamental trade-off between execution efficiency and algorithm upgradability. Solidity keeps algorithms deployable without client changes but can impose substantial execution and gas overhead, whereas Precompiled Contracts provide efficient execution only for routines that are embedded in the client and are therefore difficult to evolve independently. The central challenge is thus to replace contract-facing cryptographic implementations without sacrificing efficient execution, deterministic behavior across nodes, or compatibility with the existing EVM interface.

EvoCrypt addresses this trade-off by separating the stable contract-facing interface from replaceable WASM implementations and by separating module preparation from module activation. The stable interface preserves how smart contracts invoke cryptographic operations, while portable WASM modules allow implementations to be replaced without embedding each algorithm in contract bytecode or rebuilding the client for every change. Asynchronous preparation moves verification, compilation, and instantiation away from the transaction execution path, whereas Activation-Block-based selection ensures that nodes switch versions according to a common chain-derived rule rather than local readiness time. This separation is designed to combine efficient algorithm-level accounting with deterministic, incrementally deployable upgrades.

The main contributions of this article are summarized as follows.

- Firstly, we design a Cryptographic Coprocessor that separates contract-facing cryptographic algorithm interfaces from WASM module implementations, allowing algorithm implementations to be managed and replaced independently of the core EVM execution logic while using a narrower run-time boundary than process-local plugin loading.
- Secondly, we design an upgrade protocol that combines on-chain event coordination, off-chain asynchronous WASM preparation, and block-height-based deterministic activation.
- Finally, we implement a prototype of EvoCrypt and evaluate it on a 20-node private blockchain network. Experimental results show that algorithm-level gas pricing decouples per-invocation cost from the expanded EVM instruction count, thereby avoiding the opcode-level gas amplification observed in Solidity executions; the registered prices in the evaluated prototype are experimental parameters, so the results demonstrate the pricing mechanism and the resulting capability extension rather than a calibrated magnitude of gas savings.

The resulting claims are intentionally bounded: the evaluation concerns contract-facing cryptographic algorithms under the evaluated scenarios, does not claim that unrelated EVM transactions are unaffected during the upgrade preparation window, does not set up a separate consistency experiment, and does not treat the WASM sandbox as a formal security proof.

2 Related Work

Research on cryptographic algorithm upgrades in blockchain systems has not yet resolved the combined problem of efficient invocation and practical implementation. In this paper, we review related approaches according to the layer at which they support cryptographic algorithm upgrades, including protocol- and client-level upgrades, contract-level upgrades, and dynamic runtime updates.

Existing blockchain upgrade schemes primarily replace code at the protocol or client level, where a coordinated version transition preserves a common execution rule across nodes. Cardano’s hard-fork combinator moves the chain between protocol eras at designated upgrade points, while work on updatable blockchains studies synchronization between old and new chains [10, 4, 5]. Soft-fork approaches instead place selected policy changes within rules that can be adopted without replacing the entire protocol [23]. Phoenix extends this direction by embedding upgrade code in blockchain transactions and compiling it at runtime. Polkadot stores WASM-based runtime logic on chain and activates governance-approved replacements during block synchronization [19, 15]. These approaches support chain-wide or client-wide evolution, but they require protocol coordination and place the replacement boundary across a substantial portion of system execution.

Contract-level schemes update application logic without replacing the entire blockchain client, using deployable contracts, proxy indirection, or bytecode rewriting. EVMPatch automatically patches deployed Ethereum smart contracts through bytecode rewriting and proxy-based deployment, while Solidity implementations offer application-level flexibility within the EVM [17, 6]. LEAGAN further extends contract-level upgradeability through data separation, incremental hashing, revision control, and status-contract-based version management, enabling contract logic to evolve while preserving persistent state and supporting non-halt deployment [11]. Precompiled Contracts provide a complementary client-native path for selected cryptographic routines through reserved EVM-facing addresses [7, 16, 3]. These mechanisms avoid some forms of client redeployment. Contract-level implementations remain constrained by EVM execution cost and state or interface compatibility, whereas adding or replacing a Precompiled Contract normally requires changes to client and protocol rules.

Dynamic software-update schemes move replacement below the application layer by loading new components into a running host, with emphasis on availability, version compatibility, or isolation. KylinX supports runtime library replacement through dynamic mapping, Mvedsua combines dynamic updates with multi-version execution, PYLIVE changes Python code online, and distributed-

system research addresses modular updates and version consistency [22, 14, 9, 1, 2]. WASM provides a portable artifact format and explicit host boundary for this style of replacement. In blockchain systems, existing WASM-based upgrades generally target broad runtime logic rather than a narrow EVM-facing cryptographic interface [21, 15].

3 EvoCrypt Design

EvoCrypt is a cryptographic coprocessor framework for dynamically upgrading contract-facing cryptographic algorithms. This section presents the system model, the upgrade protocol, and the invocation protocol that together define the design. We focus on the coordination, version-selection, and contract-facing execution rules of EvoCrypt, while the prototype implementation and experimental configuration are described in Section 5.

3.1 System Model

The EvoCrypt system model comprises four interacting parts: the Community, the Blockchain, the EVM, and the Cryptographic Coprocessor. The Community consists of developers and an authorizing committee that proposes and approves algorithm upgrades; the Blockchain is the peer-to-peer network that maintains the canonical chain and shared world state, including smart contracts and the Management Contract; the EVM is the core transaction-processing module within each node and executes smart-contract transactions; and the Cryptographic Coprocessor is the node-local component that prepares, activates, and executes replaceable cryptographic algorithm modules. After the committee approves an upgrade proposal, the proposer submits it through the Management Contract interface, whereas the Invocation Protocol can be initiated through a smart-contract call processed by the EVM of any Blockchain node. The Management Contract maintains the algorithm metadata and provides the contract-facing interfaces through which authorized upgrades are recorded and algorithms are invoked, while the coprocessor uses the resulting chain state to process the corresponding modules. Figure 1 summarizes these objects and their relationships.

3.2 Upgrade Protocol

On-chain publication. On-chain publication establishes the authoritative version record that coordinates the subsequent preparation and activation stages. After the committee approves a proposal, a committee-authorized proposer submits it through the designated upgrade interface; the Management Contract admits the proposal only after the committee has reached consensus on its contents, stores the version metadata, and emits an upgrade event. The EVM path records the metadata but does not compile or instantiate the WASM module, so node-dependent preparation remains outside the transaction that changes

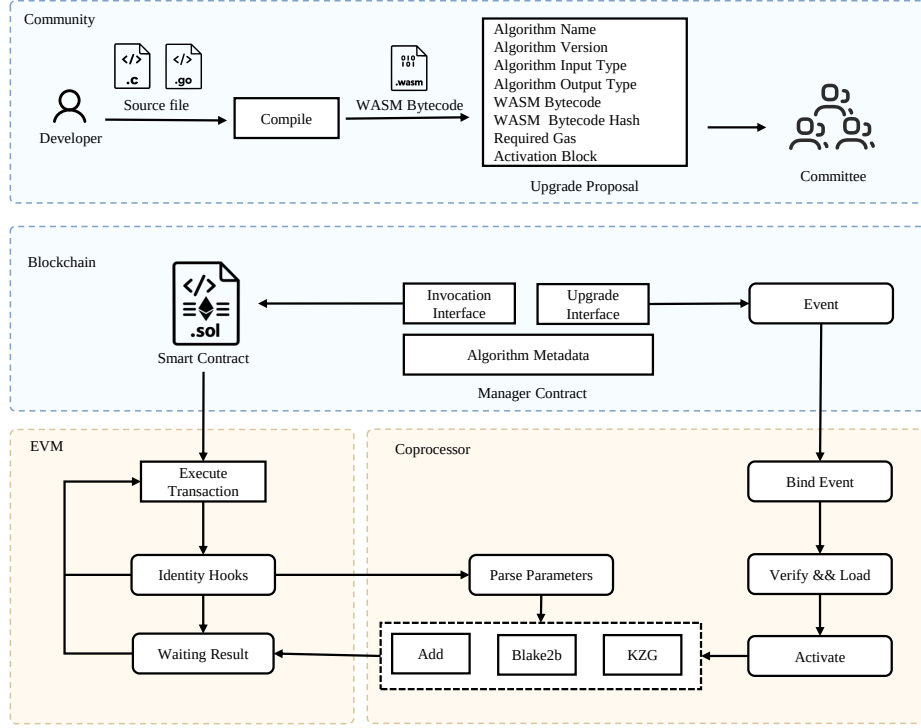


Fig. 1. System model for proposed EvoCrypt framework.

the chain state. The admitted record is the chain-authoritative description from which every node reconstructs the same version: it binds the algorithm identity and version to the WASM artifact, module hash, interface descriptors, resource parameters, and Activation Block, while the event carries only $\langle a, v, H_{a,v} \rangle$ as a notification index. Write $\mathcal{V}_a$ for the set of versions already registered for algorithm a ; the metadata binding prevents nodes from preparing different artifacts or execution rules for the same admitted upgrade.

Definition 1 (Version metadata). Let a be an algorithm name and v a version number. An admitted upgrade is the metadata record

$$\mu_{a,v} = (a, v, C_{a,v}, h_{a,v}, g_{a,v}^{(0)}, \lambda_{a,v}, \tau_{a,v}^{\text{in}}, \tau_{a,v}^{\text{out}}, H_{a,v}), \quad (1)$$

where $C_{a,v}$ is the compiled WASM bytecode, $h_{a,v} = \text{Keccak256}(C_{a,v})$ is the module hash, $g_{a,v}^{(0)}$ is the initial gas fee, $\lambda_{a,v}$ is the gas slope parameter, $\tau_{a,v}^{\text{in}}$ and $\tau_{a,v}^{\text{out}}$ are the ABI input and output type descriptors, and $H_{a,v}$ is the Activation Block height. The Management Contract stores an encoding of these fields. Each node recomputes $h_{a,v}$ from the recovered bytecode during preparation and binds that hash to the prepared version.

Algorithm 1 Activation-Block-based WASM upgrade protocol.

Require: Metadata $\mu_{a,v}$ as in (1); node set $\mathcal{N}$
Ensure: Prepared set updated on every $n_i \in \mathcal{N}$

- 1: committee-authorized proposer submits $\mu_{a,v}$ through the designated upgrade interface
- 2: Management Contract stores $\mu_{a,v}$; emits event $\langle a, v, H_{a,v} \rangle$
- 3: **for all** $n_i \in \mathcal{N}$ **do**
- 4: observe $\langle a, v, H_{a,v} \rangle$; fetch $\mu'_{a,v}$ from Management Contract
- 5: **if** event fields disagree with $\mu'_{a,v}$ **then**
- 6: reject (a, v)
- 7: **else**
- 8: decode $C_{a,v}$; set $h_{a,v} \leftarrow \text{Keccak256}(C_{a,v})$
- 9: **repeat**
- 10: compile and instantiate; validate the required module entry point
- 11: **until** preparation succeeds
- 12: persist $h_{a,v}$ and $\mu_{a,v}$; mark (a, v) prepared
- 13: **end if**
- 14: **end for**

Node-local preparation. Node-local preparation converts the admitted metadata record into executable state on each node without changing the active version. After observing the upgrade event, each client recovers $\mu_{a,v}$ through the metadata retrieval interface, verifies that the event index agrees with the stored record, decodes the artifact, and recomputes its hash. The Upgrade Handler then coordinates compilation, instantiation, and validation of the required module entry point; once these checks succeed, the node records (a, v) as prepared. This stage resolves the difference between a chain-level upgrade record and the node-local work required to execute a WASM module: preparation can proceed asynchronously, a failed attempt retries from the authoritative chain record, and the already executed upgrade transaction is not rolled back. Algorithm 1 gives the complete publication and preparation procedure.

Height-based activation. Height-based activation makes the selected version a function of shared chain height rather than node-local preparation time. The Upgrade Block is the block that includes the admitted upgrade transaction and emits $\langle a, v, H_{a,v} \rangle$; after that block, each node independently completes the off-chain Load–Verify–Ready sequence, while invocations before $H_{a,v}$ continue to use the previously selected version. At the Activation Block, nodes sharing the same chain view select version v according to the rule below, so differences in compilation time cannot produce different algorithm versions at the same execution height. If the selected version is not prepared when a call arrives, the node reloads its authoritative metadata and completes preparation before proceeding; this forced-activation policy preserves the height-based rule instead of allowing local readiness to create a version split. Figure 2 shows this separation between publication, preparation, and activation.

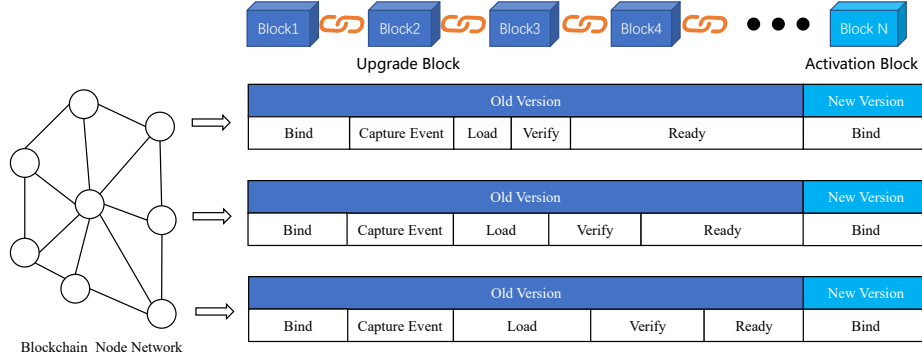


Fig. 2. Node-local preparation and deterministic network-wide activation of an EvoCrypt upgrade.

Definition 2 (Active version). Let $\mathcal{V}_a$ be the registered version set of algorithm a , and let b be the execution block height. The version used at height b is

$$v^*(a, b) = \arg \max_{v \in \mathcal{V}_a} \{(H_{a,v}, v) \mid H_{a,v} \leq b\}. \quad (2)$$

The $\arg \max$ is lexicographic on the pair $(H_{a,v}, v)$: among versions whose activation height does not exceed b , the rule prefers the latest activation height and, for a tied height, the larger version number. If the feasible set is empty, no version is active and the invocation fails with a missing-version error.

3.3 Invocation Protocol

The Invocation Protocol uses a unified Management Contract entry point so that contract developers can invoke different algorithms without changing the destination address. A caller serializes the algorithm name a and its parameters into an encoded request x and submits the request through the designated invocation interface. This arrangement simplifies off-chain monitoring because upgrade metadata and invocation-related events are associated with one on-chain management endpoint, while new algorithms can be added behind the same endpoint. The contract depends on the stable invocation boundary and the ABI descriptors of the selected version rather than on the module location, implementation language, or node-local loading state. If $v^*(a, b)$ is not yet prepared, the coprocessor prepares that version before execution rather than redirecting the request to an obsolete implementation.

The EVM-side Hooks identify requests addressed to the designated invocation interface and route them to the Cryptographic Coprocessor. Requests that do not match this interface remain on the ordinary EVM execution path, so the coprocessor handles only contract-facing cryptographic calls. After the Hooks perform this routing, the coprocessor parses (a, x) , resolves $v^*(a, b)$ by (2), and prepares the selected version if necessary. The selected algorithm module receives

Algorithm 2 Contract-to-coprocessor invocation protocol.**Require:** Recognized invocation request d , block height b , gas budget g **Ensure:** ABI-encoded result y or explicit error

```

1:  $(a, x) \leftarrow \text{Parse}(d)$ 
2: if the request is invalid then
3:   return ABI error
4: end if
5:  $v \leftarrow v^*(a, b)$  by (2)
6: if no applicable version exists then
7:   return missing-version error
8: end if
9: ensure  $(a, v)$  is prepared under the mandatory-preparation policy
10:  $G_{\text{req}} \leftarrow G(a, v, x)$  by (3)
11: if  $g < G_{\text{req}}$  then
12:   return out-of-gas error
13: end if
14: charge  $G_{\text{req}}$ ;  $y \leftarrow \text{Execute}(a, v, x)$ 
15: return  $y$ , or runtime error on failure

```

the encoded parameters and returns an encoded result; the same contract-facing call shape is preserved as long as $\tau_{a,v}^{\text{in}}$ and $\tau_{a,v}^{\text{out}}$ remain compatible.

The complete call flow resolves the version before charging and executing the algorithm. The EVM-side Hooks first identify a contract-facing cryptographic request and route it to the coprocessor; Algorithm 2 describes the processing that follows this routing. The coprocessor then parses the request, resolves the applicable version, ensures that the version is prepared, computes the algorithm-level fee, and executes the selected algorithm or returns the corresponding protocol error.

Definition 3 (Algorithm-level gas schedule). *Each algorithm version uses an independent deterministic pricing rule so that its fee can reflect the resource class of the algorithm while remaining identical across clients. A call with inner input x against version v of algorithm a is charged*

$$G(a, v, x) = g_{a,v}^{(0)} + \lambda_{a,v} \cdot |x|, \quad (3)$$

where $|x|$ is the byte length of the call-stage inner input, $g_{a,v}^{(0)}$ is the initial gas fee, and $\lambda_{a,v}$ is the gas slope parameter registered in $\mu_{a,v}$. The charged amount does not depend on node-local CPU time or on the number of WASM instructions executed inside the module.

The algorithm-level schedule provides a deterministic economic model for contract-facing cryptographic computation. Its coefficients are intended to reflect computational complexity: a more demanding algorithm can be assigned a larger fixed cost, a larger length-dependent coefficient, or both, so that its expected resource consumption is reflected in the charged fee. At the same time, the fee must remain economically reasonable for contract users rather than being tied to an

unbounded or node-specific execution measurement. Because blockchain clients must reach the same result for the same execution, the schedule cannot depend on local CPU time, runtime fluctuations, or the number of WASM instructions observed by an individual client. The fixed-plus-length form therefore combines a stable per-algorithm charge with an input-sensitive component, following the deterministic pricing intuition of Precompiled Contracts while retaining a simple way to account for input size. Nodes that resolve the same $v^*(a, b)$ and receive the same input consequently compute the same fee. The coefficients used in Section 5 are experimental parameters that demonstrate this pricing mechanism rather than a calibrated economic optimum.

4 Security Analysis

4.1 Attack Circumstances

The security analysis focuses on attack circumstances introduced by cryptographic algorithms that are upgraded at runtime and invoked from contracts. Upgrade submission is restricted to committee-authorized proposers (Section 4.2), so the analysis distinguishes two adversary positions. An external attacker may craft transaction inputs and exert timing pressure around node-local upgrade preparation but cannot register artifacts or metadata fields. A faulty or compromised admitted proposer may influence uploaded artifacts and metadata fields. We also consider erroneous upgrade artifacts and local preparation failures because they can lead to the same safety outcomes as adversarial inputs. The analysis does not model a compromised Geth binary, a broken consensus protocol, a captured committee, or cryptographic primitives whose mathematical assumptions are false.

Malicious or malformed WASM artifacts. A faulty or compromised admitted proposer may submit a WASM module that is malformed, omits the required entry point, imports unauthorized host functions, declares unsafe memory behavior, or attempts to exploit the WASM runtime boundary. This scenario targets execution isolation: the module is dynamically supplied after client deployment, so the client cannot rely on source-level review or static compilation into the Geth binary.

Inconsistent upgrade metadata. An attacker or faulty proposer may publish metadata whose version, Activation Block, gas parameters, ABI type descriptors, event projection, or encoded artifact are inconsistent with the intended upgrade. Such inconsistencies can cause version confusion, incorrect ABI decoding, incorrect gas charging, or preparation of an artifact that does not correspond to the version later selected during EVM execution.

Resource-exhaustion attempts. Dynamic algorithm loading creates denial-of-service risks. A module may be large, slow to compile, memory intensive, or designed to run for a long time. A low gas setting may underprice an expensive cryptographic operation, and large uploaded artifacts may increase transaction-input, storage, propagation, and data-availability pressure.

Asynchronous readiness divergence. Nodes may observe the same on-chain upgrade event but complete WASM validation, compilation, instantiation, and local persistence at different wall-clock times. A node may also fail to prepare the required artifact. If local readiness time or local wall-clock time were used for version selection, nodes executing the same block could resolve the same contract-facing call to different algorithm implementations.

4.2 Security Solutions

The proposed coprocessor addresses these attack circumstances through committee-gated upgrade admission, sandboxed WASM execution, authoritative metadata recovery, deterministic resource accounting, and Activation-Block-based version selection. These controls reduce the security boundary exposed by dynamic upgrades, but they are not a formal proof of sandbox security against all adversarial modules, governance failures, or misconfigured deployments.

Committee-gated upgrade admission. The upgrade interface is not open to arbitrary accounts. The Management Contract admits a versioned proposal only through the committee-authorized path: a proposal becomes effective only after the committee reaches consensus on the algorithm name, version, encoded artifact, invocation gas metadata, ABI descriptors, and Activation Block height. This access restriction removes the unauthorized-proposer cases from the malicious-artifact and metadata-inconsistency circumstances, because an outside attacker cannot register an artifact, a gas price, or an activation height without committee approval. The residual risk is a faulty, compromised, or captured committee together with misconfigured admission rules; these governance failures remain outside the analyzed boundary, and a production deployment would require concrete committee rotation, quorum, and key-management rules.

Sandboxed WASM execution. WASM narrows the execution boundary compared with a process-local plugin or shared-library interface. The module runs inside a WASM Runtime with linear memory, explicit exports, and validated imports. In the prototype, modules are invoked through a fixed module entry point, and imported memories or unauthorized host imports are rejected. This design reduces direct access from the algorithm module to Geth process memory, local files, system networking, and host functions that are not explicitly exposed. A production deployment would still require stronger artifact admission policies, runtime diversity testing, reproducible builds, and governance rules for approving cryptographic modules.

Authoritative metadata recovery. The Management Contract is treated as the authoritative source of upgrade metadata. The emitted event is used as a notification and index, while the Event Listener recovers the full metadata through the designated metadata retrieval interface and checks the observed event fields against the contract state. The recovered WASM bytes are hashed locally and bound to the prepared version metadata, and the ABI descriptors determine how inputs and outputs are decoded and encoded. This binding reduces the risk that a node prepares an artifact or ABI interface different from the version selected during execution.

Deterministic resource accounting. Invocation gas is determined by the algorithm-level schedule (3) registered in $\mu_{a,v}$, not by EVM metering of the module’s execution logic. Every node resolving the same $v^*(a, b)$ on the same inner input x evaluates the same pair $(g_{a,v}^{(0)}, \lambda_{a,v})$ from the Management Contract, so the consistency of gas charging reduces to the consistency of the contract state, which the underlying consensus protocol already provides: nodes cannot disagree on $G(a, v^*(a, b), x)$ without disagreeing on the chain itself. The design further bounds module memory, applies an execution budget, validates imports and required exports, and separates module preparation from the upgrade transaction’s EVM execution path. These measures reduce the immediate attack surface of dynamic loading. They do not fully solve economic pricing: the current prototype invocation gas values are experimental parameters, and future work must calibrate call-path gas pricing against execution cost rather than treating upgrade-upload transaction cost as part of the coprocessor pricing model. Accordingly, the evaluation in Section 5 demonstrates the pricing mechanism itself, namely that invocation cost is decoupled from the expanded EVM instruction count, together with the metered Solidity comparison side; the claimed benefit is this capability extension, not a validated magnitude of gas savings.

Forced upgrade at the Activation Block. Upgrade consistency is treated as a protocol security property. Version selection follows (2): it depends on the execution block height rather than local wall-clock time or local readiness time. Therefore, nodes with the same chain view compute the same $v^*(a, b)$. Once that version applies, a node must not silently fall back to an obsolete implementation. If local preparation is incomplete when the Activation Block is reached or when a requiring call is executed, the node forcibly reloads $\mu_{a,v^*(a,b)}$ from authoritative metadata and completes preparation before serving the new version. This paper does not set up a separate consistency experiment; the claim remains a protocol analysis rather than node-level evidence for state-root equality, arbitrary network safety, or behavior when forced preparation cannot complete.

5 Evaluation

The evaluation asks whether the proposed coprocessor can be used as a practical execution path for upgradable contract-facing cryptographic algorithms. Following the target outline, we focus on upgrade efficiency and execution efficiency. Upgrade consistency is discussed as a protocol security property in Section 4.2 and is not set up as a separate experiment.

The prototype used in these experiments was implemented in Geth by adding the Management Contract interface, EVM call recognition for the contract-facing entry point, the Cryptographic Coprocessor routing component, the activation service, version metadata storage, and the WASM Runtime path. The invocation path recognizes contract-facing requests, calculates required gas from the registered metadata, parses the encoded request, selects the applicable algorithm version for the execution block height, passes the encoded parameters to the selected WASM module, and returns its encoded output to the EVM.

The upgrade path is event-driven. The prototype provides both a legacy unversioned upload path and a versioned upgrade path carrying activation metadata. The Event Listener observes the emitted upgrade event, and the activation service decodes the uploaded WASM artifact, computes its hash, records runtime metadata, compiles and instantiates the module through the WASM Runtime, and persists the prepared-version metadata. The WASM Runtime is wazero v1.12.0 and uses a memory limit, an execution budget, import validation, and a module entry-point check. These checks define implementation safeguards for the evaluated prototype boundary, not a substitute for a complete production security policy.

5.1 Implementation

Experiments were conducted on a Geth-based private Clique network using a modified Geth v1.17.5-unstable, built with Go 1.25.7. Solidity contracts were compiled with solc 0.8.26; WASM modules were built with TinyGo 0.42.0. Host hardware specifications remain to be recorded for full reproducibility.

We report *network-wide upgrade duration* as control-client observable Geth-side setup time from upgrade submission until all evaluated nodes finish upgrade handling and the new algorithm is callable. This interval includes RPC submission, receipt observation, event handling, and node-local WASM preparation on each node, but excludes network propagation beyond the measured RPC path, consensus delay, and block-production waiting. With `period=0`, the network does not enforce an inter-block interval, so the reported duration reflects client-side upgrade processing rather than Clique block spacing.

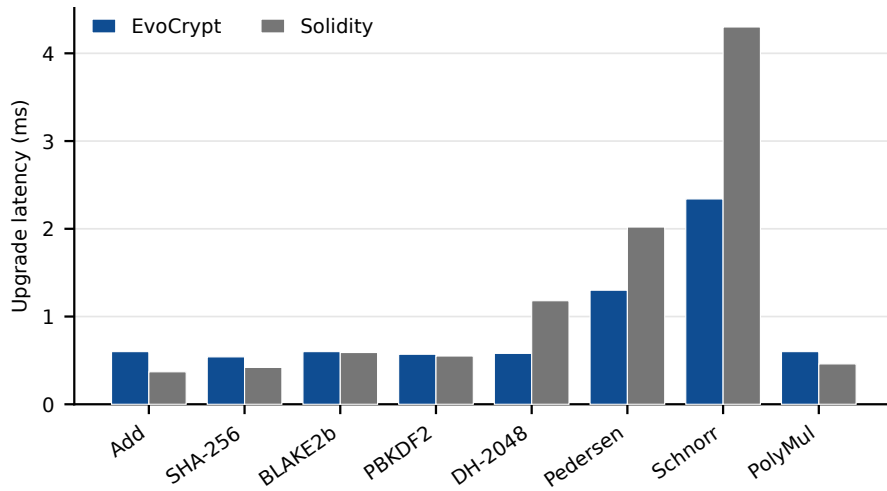
Table 1 summarizes the eight evaluated contract-facing algorithms, their implementation artifact sizes on the three benchmark paths, and the cryptographic role of each workload. Go (B) reports the byte size of the Native reference implementations; WASM (B) reports the uploaded module payloads compiled from the same reference logic; Solidity (B) reports on-chain benchmark contract source size before deployment. For Dh2048Secret and PedersenCommit, the listed Solidity sizes correspond to algorithm-specific wrappers that link a shared modular-arithmetic library (8,524 B); deployed bytecode and invocation `gasEstimate` in Section 5.3 reflect the fully linked contracts. Add is a non-cryptographic baseline; the remaining workloads cover hash functions, password-based key derivation, key exchange, commitments, signature verification, and finite-field polynomial multiplication.

For upgrade efficiency, we compare the coprocessor upgrade upload path with Solidity contract deployment using mean setup latency. For execution efficiency, we compare three paths after setup has completed: the WASM-backed upgrade path (EvoCrypt), the Native algorithm, and the Solidity Contract. The Native algorithm baseline uses the same Go reference implementations as in Table 1, executed in-process in the evaluation client to measure execution latency outside the EVM rather than through fixed mainnet precompile addresses. Invocation `gasEstimate` denotes the estimated gas returned for the contract-facing invocation and is not receipt-level `gasUsed`.

Table 1. Information on the evaluated algorithms.

Algorithm	Go (B)	WASM (B)	Solidity (B)	Role
Add	156	11,287	181	Non-cryptographic baseline
Sha256	210	68,288	219	Cryptographic hash
Blake2bSum256	4,911	8,724	5,340	Cryptographic hash
Pbkdf2Sha256	1,377	72,580	2,269	Key derivation
Dh2048Secret	1,595	45,919	473 [†]	Key exchange
PedersenCommit	1,871	95,780	645 [†]	Commitment scheme
SchnorrVerify	2,171	95,960	962	Signature verification
PolynomialMul	790	7,987	953	Algebraic operation

[†]Solidity source size for the algorithm contract only; modular arithmetic is provided by the shared library linked at deployment.

**Fig. 3.** Setup latency comparison between EvoCrypt upgrade and Solidity contract deployment.

5.2 Upgrade Efficiency

The upgrade-efficiency experiment compared the latency of making a cryptographic algorithm callable through the coprocessor upgrade path against deploying an equivalent Solidity implementation. Figure 3 reports mean setup latency across all eight evaluated algorithms. The Native algorithm baseline is omitted from this comparison because it requires no separate upload or deployment step in the evaluation setup.

On lightweight algorithms, the two setup paths remained close in latency. For Add, SHA-256, BLAKE2b, and PBKDF2, mean upgrade latency stayed below 0.6 ms for EvoCrypt and below 0.6 ms for Solidity, with the largest gap on the

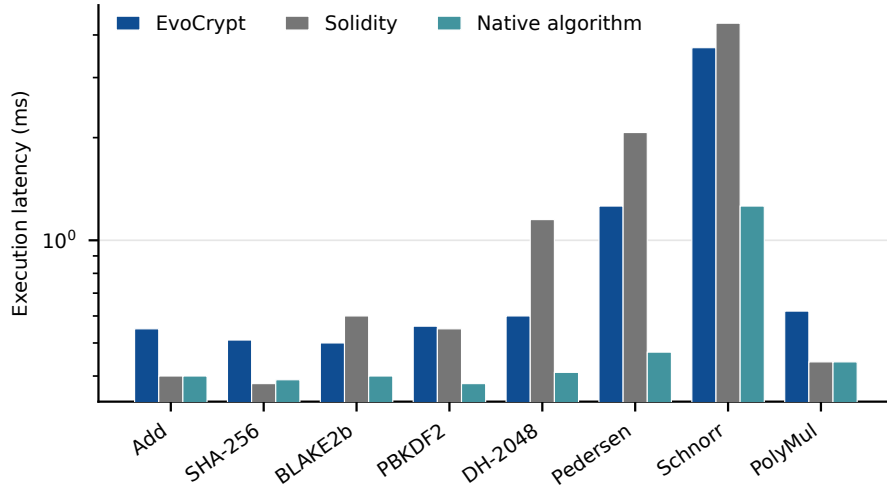


Fig. 4. Execution latency comparison among EvoCrypt, Solidity contract execution, and Native algorithm.

non-cryptographic Add baseline (0.60 ms versus 0.37 ms). As algorithm complexity increased, EvoCrypt reduced setup latency relative to contract deployment. For DH-2048, PedersenCommit, and SchnorrVerify, EvoCrypt required 0.58–2.34 ms, whereas Solidity required 1.18–4.30 ms. The largest relative gain appeared on SchnorrVerify, where the upgrade path completed in 2.34 ms compared with 4.30 ms for contract deployment. These latency results report network-wide upgrade duration as defined in Section 5.1: Geth-side handling through all evaluated nodes, excluding network propagation and consensus delay. They do not include upgrade-window availability for unrelated EVM traffic.

5.3 Execution Efficiency

The execution-efficiency experiment evaluated steady-state invocation cost after each algorithm had been made callable. Figure 4 compares mean `eth_call` latency across EvoCrypt, Solidity, and Native algorithm paths, and Figure 5 compares `gasEstimate` for EvoCrypt and Solidity. Each path was measured after warm-up calls using repeated `eth_call` samples on the evaluated private-chain prototype.

Native algorithm remained the fastest path on every algorithm, with mean latency between 0.38 ms (PBKDF2) and 1.26 ms (SchnorrVerify). EvoCrypt tracked Native algorithm closely on lightweight workloads: for Add, SHA-256, BLAKE2b, PBKDF2, and PolynomialMul, EvoCrypt latency remained within 0.50–0.62 ms, whereas Solidity stayed within 0.38–0.60 ms on the same set. On heavier cryptographic workloads, EvoCrypt reduced invocation latency relative

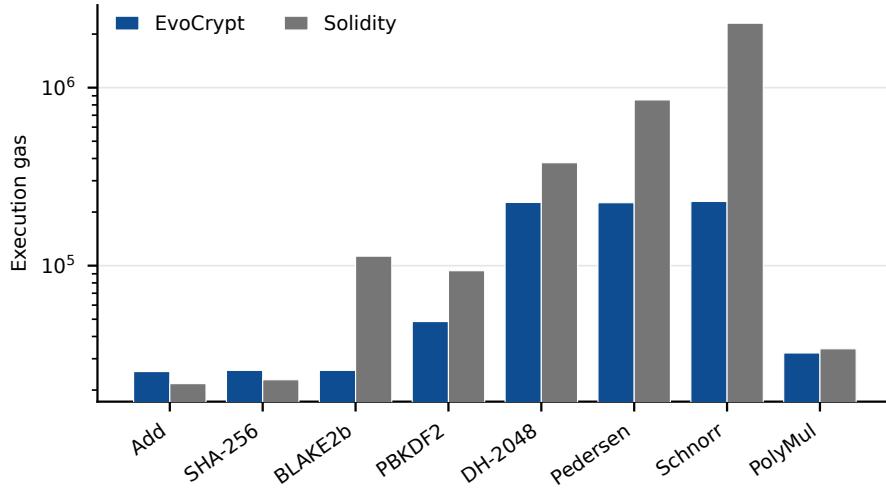


Fig. 5. Invocation `gasEstimate` comparison between EvoCrypt and Solidity contract execution.

to Solidity while remaining above Native algorithm. For DH-2048, PedersenCommit, and SchnorrVerify, EvoCrypt required 0.60, 1.26, and 3.67 ms, respectively, compared with 1.15, 2.07, and 4.33 ms through Solidity and 0.41, 0.47, and 1.26 ms through Native algorithm. SchnorrVerify therefore remained the clearest tail case: although EvoCrypt improved on Solidity, it still exceeded Native algorithm by roughly $2.9\times$ on mean `eth_call` latency.

The gas comparison makes the amplification effect directly visible. Solidity executions pay for the expanded EVM instruction sequence, so their estimates grew sharply with algorithmic complexity: Blake2bSum256, PedersenCommit, and SchnorrVerify required 113,059, 853,145, and 2,301,691 gas through contract execution. EvoCrypt instead charges the registered algorithm-level price, and its estimates for the same workloads were 25,801, 225,695, and 229,437 gas, up to an order of magnitude lower. For Add, SHA-256, and PolynomialMul, whose Solidity expansions remain short, the two contract-facing paths stayed within 13–15% of each other, which is consistent with the amplification account: the separation between the two paths appears exactly where the instruction expansion grows. These numbers describe the current prototype invocation pricing model only; they are not receipt-level `gasUsed` measurements, and the registered prices are experimental parameters rather than a calibrated schedule. The comparison therefore demonstrates the decoupling mechanism and meters the Solidity side, rather than establishing a specific magnitude of gas savings.

To assess the applicability boundary of the coprocessor, we conducted a separate stress experiment on computationally intensive cryptographic algorithms. This experiment is distinct from the eight algorithms evaluated above: the se-

Table 2. Execution of computationally intensive cryptographic algorithms.

Algorithm	EvoCrypt (ms)	Precompiled Contract (ms)	WASM (B)
Aigis-Sig2	1.299	1.030	72,213
Dilithium3	1.589	1.100	26,679
ML-DSA-65	1.565	1.052	70,422
SLH-DSA-SHAKE-192f	19.458	6.241	19,339
Groth16/BLS12-381	156.138	6.227	139,279
Groth16/BN254	25.532	3.172	580,975

lected workloads include post-quantum signatures and a pairing-based zero-knowledge proof system, which are substantially more demanding than the regular contract-facing benchmark suite. Table 2 reports the measured invocation time for EvoCrypt and the corresponding Precompiled Contract implementation, together with the size of the uploaded WASM module. The execution times are reported in milliseconds and the module sizes in bytes.

EvoCrypt successfully executes all workloads in this stress experiment, including algorithms whose arithmetic complexity and implementation size make direct Solidity realization impractical, and which are not available through the existing Precompiled Contract catalog. The measurements show the expected trade-off: the coprocessor is slower than client-native execution, with EvoCrypt latency ranging from $1.3\text{--}25.1\times$ the corresponding Precompiled Contract latency across the reported cases. Nevertheless, the cost remains acceptable for workloads for which an ordinary contract implementation is unavailable or would incur prohibitive EVM instruction expansion. These results therefore support the main capability claim of EvoCrypt: it can expose cryptographic algorithms beyond the practical contract implementation boundary while retaining a stable contract-facing interface. Precompiled Contracts remain the preferable path when an algorithm is already standardized and embedded in the client, but EvoCrypt provides a deployable alternative for complex algorithms that would otherwise require a client modification and coordinated network upgrade.

6 Conclusion

This paper addressed how smart contracts can access evolving cryptographic capabilities without embedding complex algorithms in EVM bytecode or requiring a client modification for every update. EvoCrypt separates a stable contract-facing interface from replaceable client-side WASM implementations, enabling algorithms to evolve independently while preserving deterministic contract invocation and explicit algorithm-level resource accounting.

The evaluated prototype shows that EvoCrypt reduces upgrade and invocation costs relative to contract-level implementations, while supporting computationally intensive algorithms that are impractical to implement directly in Solidity. Although its execution remains slower than Precompiled Contracts,

the measured overhead is an acceptable trade-off for extending cryptographic capability beyond the fixed client catalog. EvoCrypt therefore supports the evolution of smart-contract-side cryptographic capabilities, lowers invocation cost, and reduces the difficulty of developing cryptographic algorithms for blockchain applications. Future work will refine resource pricing and evaluate broader deployment and security conditions.

References

1. Ajmani, S., Liskov, B., Shrira, L.: Modular software upgrades for distributed systems. In: Thomas, D. (ed.) ECOOP 2006 – Object-Oriented Programming. Lecture Notes in Computer Science, vol. 4067, pp. 452–476. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11785477_26
2. Baresi, L., Ghezzi, C., Ma, X., Panzica La Manna, V.: Efficient dynamic updates of distributed components through version consistency. *IEEE Transactions on Software Engineering* **43**(4), 340–358 (2017). <https://doi.org/10.1109/TSE.2016.2592915>
3. Buterin, V., Reitwiessner, C.: EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128. *Ethereum Improvement Proposals*, no. 197. <https://eips.ethereum.org/EIPS/eip-197> (2017), final. Accessed 21 August 2026
4. Cardano Foundation: About hard forks. <https://docs.cardano.org/about-cardano/evolution/about-hard-forks>, accessed 15 September 2026
5. Ciampi, M., Karayannidis, N., Kiayias, A., Zindros, D.: Updatable blockchains. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) *Computer Security – ESORICS 2020. Lecture Notes in Computer Science*, vol. 12309, pp. 590–609. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-59013-0_29
6. Ethereum.org: Ethereum virtual machine (EVM). <https://ethereum.org/developers/docs/evm/>, accessed 21 August 2026
7. EVM Codes: Precompiled contracts. <https://www.evm.codes/precompiled>, accessed 21 August 2026
8. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pp. 212–219. Association for Computing Machinery, New York, NY, USA (1996). <https://doi.org/10.1145/237814.237866>
9. Huang, H., Xiang, C., Zhong, L., Zhou, Y.: PYLIVE: On-the-fly code change for python-based online services. In: *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 349–363. USENIX Association, Berkeley, CA, USA (2021), <https://www.usenix.org/conference/atc21/presentation/huang>
10. Input Output: Combinator makes easy work of hard forks. <https://iohk.io/en/blog/posts/2020/07/23/combinator-makes-easy-work-of-hard-forks/> (2020), accessed 15 September 2026
11. Kumar, G., Saha, R., Conti, M., Buchanan, W.J.: LEAGAN: A decentralized version-control framework for upgradeable smart contracts. *IEEE Transactions on Services Computing* **18**(3), 1529–1542 (2025). <https://doi.org/10.1109/TSC.2025.3562323>
12. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf> (2008), accessed 21 August 2026

13. National Institute of Standards and Technology: Post-quantum cryptography. <https://csrc.nist.gov/projects/post-quantum-cryptography> (2026), updated 5 August 2026. Accessed 21 August 2026
14. Pina, L., Andronidis, A., Hicks, M., Cadar, C.: MVEDSUA: Higher availability dynamic software updates via multi-version execution. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 573–585. Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3293882.3330551>
15. Polkadot: Runtime upgrades. <https://docs.polkadot.com/parachains/maintain/runtime-upgrades/>, accessed 15 September 2026
16. Reitwiessner, C.: EIP-196: Precompiled contracts for addition and scalar multiplication on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 196. <https://eips.ethereum.org/EIPS/eip-196> (2017), final. Accessed 21 August 2026
17. Rodler, M., Li, W., Karame, G.O., Davi, L.: EVMPatch: Timely and automated patching of ethereum smart contracts. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 1289–1306. USENIX Association, Berkeley, CA, USA (2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/rodler>
18. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review* **41**(2), 303–332 (1999). <https://doi.org/10.1137/S0036144598347011>
19. Wang, C., Li, P., Fan, X., Tang, Z., Zeng, Y., Sakurai, K.: Phoenix: A live upgradable blockchain client. *IEEE Transactions on Sustainable Computing* **8**(4), 703–714 (2023). <https://doi.org/10.1109/TSUSC.2023.3282586>
20. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf> (2014), ethereum Yellow Paper. Accessed 21 August 2026
21. World Wide Web Consortium: WebAssembly Core Specification. <https://www.w3.org/TR/wasm-core-2/>, w3C Candidate Recommendation Draft. Accessed 3 September 2026
22. Zhang, Y., Crowcroft, J., Li, D., Zhang, C., Li, H., Wang, Y., Yu, K., Xiong, Y., Chen, G.: KylinX: A dynamic library operating system for simplified and efficient cloud virtualization. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18). pp. 173–186. USENIX Association, Berkeley, CA, USA (2018), <https://www.usenix.org/conference/atc18/presentation/zhang-yiming>
23. Zindros, D.: Soft power: Upgrading chain macroeconomic policy through soft forks. In: Bernhard, M., Bracciali, A., Gudgeon, L., Haines, T., Klages-Mundt, A., Matsuo, S., Perez, D., Sala, M., Werner, S. (eds.) *Financial Cryptography and Data Security. FC 2021 International Workshops. Lecture Notes in Computer Science*, vol. 12676, pp. 467–481. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-662-63958-0_36
24. Zou, W., Lo, D., Kochhar, P.S., Le, X.B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering* **47**(10), 2084–2106 (2021). <https://doi.org/10.1109/TSE.2019.2942301>